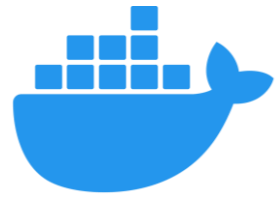


Docker Kompositionen



Beschreibung

Dieses Dokument bietet den Einstieg in Kompositionen von zusammenarbeitenden Images über das Docker-Compose Verfahren. Die Images werden dabei nicht mit einzelnen Befehlen heruntergeladen, sondern über die Docker-Compose-Datei gesteuert.

Inhaltsverzeichnis

1	ELEMENTE EINER KOMPOSITION	2
1.1	DOCKER NETWORKS	2
1.2	DOCKER VOLUMES.....	4

1 Elemente einer Komposition

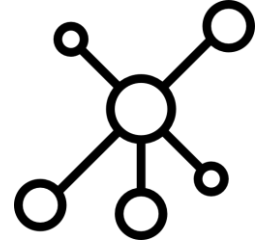
1.1 Docker Networks

Bei Docker-Compose gibt es verschiedene Möglichkeiten, Netzwerke für Container zu konfigurieren. Im Folgenden sind einige wichtige Informationen zum Thema Netzwerk in Docker-Compose aufgeführt:

Standard-Netzwerkkonfiguration

Wenn Sie in Ihrer Docker-Compose-Datei keine Netzwerkkonfiguration angeben, verwendet Docker-Compose standardmässig ein isoliertes Bridge-Netzwerk für alle Container in Ihrer Komposition.

Jeder Container hat eine eigene IP-Adresse im Netzwerk, und Container können nur über Port-Veröffentlichungen von aussen aufgerufen werden.



Externe Netzwerke

Bei der Erstellung einer Docker-Compose-Komposition wird ein gemeinsames Netzwerk für alle Container in der Komposition erstellt. Dieses Netzwerk wird standardmässig als "default" bezeichnet, kann aber auch mit einem benutzerdefinierten Namen konfiguriert werden.

Es kann aber auch ein benutzerdefiniertes Netzwerk erstellt werden, um Container in einem Netzwerk zu isolieren und eine bessere Netzwerksicherheit zu gewährleisten. Externe Netzwerke können in der Docker-Compose-Datei unter dem «networks» Abschnitt definiert werden.

Benutzerdefiniertes Netzwerk

Sie können ein benutzerdefiniertes Netzwerk für die Container in Ihrer Docker-Compose-Komposition definieren. Dadurch können Container direkt miteinander kommunizieren, ohne dass Port-Veröffentlichungen erforderlich sind. Das benutzerdefinierte Netzwerk kann über die Docker-CLI oder direkt in der Docker-Compose-Datei definiert werden.

Beispiel für ein benutzerdefiniertes Netzwerk in Docker Compose:

```
version: "3"
services:
  web:
    image: nginx
    networks:
      - my_network
  db:
    image: mysql
    networks:
      - my_network
    environment:
      MYSQL_ROOT_PASSWORD: example
networks:
  my_network:
    driver: bridge
```

In diesem Beispiel wird ein benutzerdefiniertes Netzwerk namens "my_network" erstellt. Wir definieren zwei Services: "web" und "db". Beide Services werden dem "my_network" Netzwerk zugewiesen. Der "web"-Service verwendet das Nginx-Image und der "db"-Service verwendet das MySQL-Image.

Unter dem Abschnitt `networks` definieren wir das benutzerdefinierte Netzwerk `"my_network"`. Hier können Sie mehrere Netzwerke wie auch Netzwerkoptionen konfigurieren. In diesem Beispiel verwenden wir den Standardtreiber `"bridge"`. Andere mögliche Treiber sind `"overlay"` und `"macvlan"`.

Mit diesem benutzerdefinierten Netzwerk können Container, die diesem Netzwerk zugewiesen sind, direkt miteinander kommunizieren, ohne dass Port-Veröffentlichungen erforderlich sind. Der `"web"`-Service kann auf den `"db"`-Service über den Netzwerknamen `"db"` zugreifen, anstatt auf eine IP-Adresse und einen Port zu verweisen.

Netzwerkverbindungen:

Sie können Netzwerkverbindungen bzw. Abhängigkeiten zwischen verschiedenen Containern innerhalb einer Docker-Compose-Komposition herstellen, indem Sie das `links` oder `«depends_on»` Schlüsselwort in der Docker-Compose-Datei verwenden.

Externe Öffnungen vermeiden:

Da die Container innerhalb der Komposition mit Netzwerken verbunden sind, können externe «Port-Öffnungen» weggelassen werden, wie beispielsweise bei einem Datenbank-Container.

```
ports:-  
- "3306:3306"
```

Befehle im Zusammenhang mit Docker-Compose Netzwerken:

Hier sind einige nützliche Befehle Docker-CLI Befehle, um Informationen zu Netzwerken abzurufen.

```
$ docker network ls
```

Um Informationen zu einem bestimmten Netzwerk abzurufen, verwenden Sie den folgenden Befehl:

```
$ docker network inspect <network-name>
```

Um das Netzwerk zu finden, das von einem bestimmten Container verwendet wird, können Sie den folgenden Befehl verwenden:

```
$ docker inspect {container-name-or-id}
```

1.2 Docker Volumes

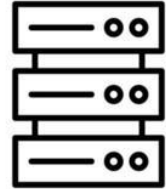
Sehr oft verwenden Images bzw. Container Volumes für die dauerhafte Speicherung von Daten. Volumes können in einer Docker-Compose Datei Volumes erstellt werden. Hierfür gibt es zwei unterschiedliche Wege.

Beispiel für die Erzeugung eines Volumes in Docker Compose:

```
version: "3"

services:
  web:
    image: nginx
    volumes:
      - my_volume:/usr/share/nginx/html

volumes:
  my_volume:
```



In diesem Beispiel erstellen wir einen Docker-Service namens "web", der das Nginx-Image verwendet. Wir definieren auch ein Volume namens "my_volume", das der Service verwendet. Das Volume wird unter dem Abschnitt volumes definiert.

Das Volume wird in diesem Beispiel im Verzeichnis /usr/share/nginx/html des Containers gemountet. Jede Änderung, die in diesem Verzeichnis gemacht wird, wird im Volume gespeichert. Wenn der Container gestoppt oder gelöscht wird, bleiben die Daten im Volume erhalten. Mit diesem Volume können auch Daten zwischen Containern geteilt werden, indem wir das gleiche Volume für mehrere Container verwenden.

Um alle Volumes aufzulisten, die auf Ihrem System erstellt wurden, können Sie den folgenden Docker-CLI-Befehl ausführen. Dieser Befehl gibt eine Liste aller auf dem System verfügbaren Volumes zurück.

```
$ docker volume ls
```

Um das Volume zu finden, das von einem bestimmten Container verwendet wird, können Sie den folgenden Befehl verwenden. Dieser Befehl gibt eine Liste aller Mounts zurück, die von dem angegebenen Container verwendet werden.

```
$ docker inspect {container-name-or-id}
```